

agente de análisis de datos nativo en MCP (Model Context Protocol)

propósito fundamental es actuar como un **analista de datos conversacional autónomo** que se conecta de forma directa y segura a los almacenes de datos (*data warehouses*) de las empresas.

¿Qué hace exactamente este Agente?

A continuación, se detalla de forma clara qué es lo que hace este agente y la tecnología que utiliza para lograrlo:

Cuando un usuario le hace una pregunta de negocio en lenguaje natural (por ejemplo, *"Muéstrame un ranking de mis productos más vendidos con su stock actual"*), el agente ejecuta un ciclo analítico completo en menos de 60 segundos :

1. **Lee y entiende el esquema** (*schema*) de la base de datos.
2. **Escribe el código SQL** idóneo para responder a la consulta.
3. **Ejecuta la consulta** de manera segura en el almacén de datos.
4. **Renderiza gráficos interactivos** y tablas con los resultados.
5. **Redacta una narrativa explicativa** (un resumen ejecutivo en prosa) lista para ser enviada al equipo.

¿Cómo lo hace? La arquitectura de un Agente Nativo en MCP

Para que este proceso sea viable en entornos corporativos reales sin comprometer la seguridad ni "alucinar", la estructura de su tecnología reside sobre cuatro pilares operativos:

- **Diagrama 1 ("Under the hood"):** Es el mapa de la **autopista** (muestra cómo entra el prompt, pasa por el agente, interactúa con el almacén vía MCP y genera los reportes finales).
- **Diagrama 2 ("Not prompts. Context"):** Es el plano interno del **motor** (cómo se gestiona la memoria, cómo se cargan las herramientas y cómo se optimizan los recursos computacionales para que el viaje no sea lento ni costoso para la empresa).

Diagrama 3 (Automation): Muestra los **músculos**. Cómo pasa de responder preguntas en un chat a convertirse en un miembro activo de la empresa que vigila tus datos por la noche, toma decisiones y delega tareas en tus herramientas de trabajo diarias.

Diagrama 4 (Gobernanza y Transparencia - SQL Studio): Le da el control final a los humanos (analistas y DBAs), permitiéndoles auditar el código SQL de la IA, modificarlo con un control de versiones y promoverlo a métricas de producción en un clic.

- **Diagrama 1 ("Under the hood"):** Es el mapa de la **autopista** (muestra cómo entra el prompt, pasa por el agente, interactúa con el almacén vía MCP y genera los reportes finales).

ilustra de forma visual el flujo técnico e interno de cómo funciona **Pulse Agent** de Wilab.

Muestra cómo el agente no es simplemente un chatbot con respuestas preprogramadas, sino un motor de toma de decisiones dinámico que interactúa directamente con tus bases de datos mediante el protocolo MCP (Model Context Protocol).

A continuación, se detalla el recorrido de la información paso a paso, de izquierda a derecha:

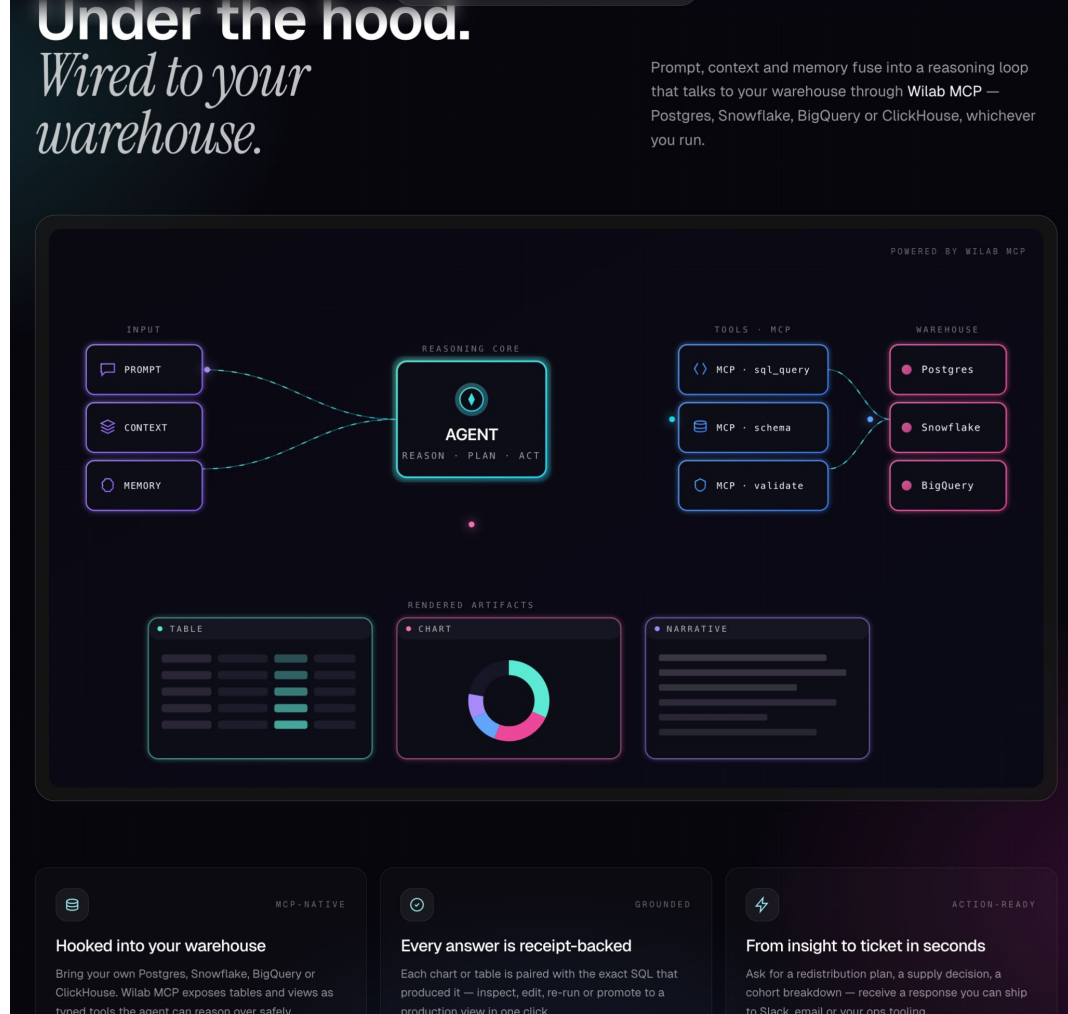
1. Las Entradas (Input)

El agente inicia su proceso fusionando tres componentes de entrada clave para dar forma al pensamiento del modelo :

- **Prompt (Instrucción):** La pregunta en lenguaje natural que hace el usuario (por ejemplo, "¿Cuáles son mis productos con stock crítico?").
- **Context (Contexto):** Información relevante del entorno de la empresa y del negocio que se carga dinámicamente según la consulta.
- **Memory (Memoria):** Compuesta por distintos niveles (memoria de corto plazo para la sesión activa, largo plazo para entidades y un grafo de conocimiento sobre el catálogo de datos). Esto permite que el agente recuerde interacciones pasadas y mejore su precisión en cada sesión.

2. El Núcleo de Razonamiento (Reasoning Core)

- En el centro se ubica el **Agent (Agente)**.
- Aquí es donde ocurre la orquestación: el agente procesa las entradas y entra en un bucle continuo de **Razonar, Planificar y Actuar** (*Reason, Plan, Act*). En lugar de generar código SQL de inmediato o de forma improvisada, primero analiza qué información necesita, diseña un plan de ejecución y luego decide qué herramientas utilizar.



3. Herramientas MCP y Almacén de Datos (Tools · MCP / Warehouse)

Para interactuar de forma segura con las bases de datos corporativas del cliente —ya sea **Postgres**, **Snowflake** o **BigQuery** (así como ClickHouse)— el agente utiliza herramientas de software específicas expuestas de manera nativa por el protocolo MCP de Wilab :

- **MCP · schema:** Lee de forma segura la estructura, metadatos y relaciones lógicas de las tablas de la base de datos sin extraer registros confidenciales.
- **MCP · sql_query:** Escribe y ejecuta la consulta SQL exacta en el motor del almacén de datos del cliente.
- **MCP · validate:** Verifica y valida que la consulta generada sea correcta, óptima y libre de errores antes de proceder al renderizado.

4. Artefactos Renderizados (Rendered Artifacts)

Una vez ejecutada la consulta SQL y procesados los datos reales del almacén del cliente, el sistema genera de forma automática tres salidas en una sola pasada :

- **Table (Tabla):** Los datos puros estructurados y ordenados para su revisión.
- **Chart (Gráfico):** La visualización interactiva idónea generada de manera automática (como el gráfico circular o de dona que se aprecia en el diagrama).
- **Narrative (Narrativa):** La explicación escrita en lenguaje natural que resume el análisis y los hallazgos clave de manera ejecutiva.

Los Tres Pilares Operativos (Sección Inferior del Gráfico)

El diagrama se complementa en la base con los tres principios de diseño bajo los cuales opera esta tecnología :

1. **Conectado a tu Almacén de Datos (MCP-Native / "Hooked into your warehouse"):** No necesitas migrar tus datos a servidores externos ni construir costosas tuberías de datos (*pipelines*). Wilab MCP expone tus bases de datos como herramientas tipadas sobre las cuales el agente razona con total seguridad y soberanía.
2. **Respaldado por Recibos (Grounded / "Every answer is receipt-backed"):** Se elimina la opacidad de la IA. Cada gráfico o tabla incluye de forma transparente el código SQL exacto que lo generó, permitiendo a los analistas inspeccionar, editar o promover esa consulta a producción en un clic dentro del SQL Studio de Wilab.
3. **Listo para la Acción (Action-Ready / "From insight to ticket in seconds"):** El flujo no termina en la visualización pasiva. El agente es capaz de tomar los resultados y enviarlos directamente a canales de Slack, correos electrónicos o crear automáticamente tareas operativas en herramientas como Linear para solucionar los problemas detectados en tiempo real.

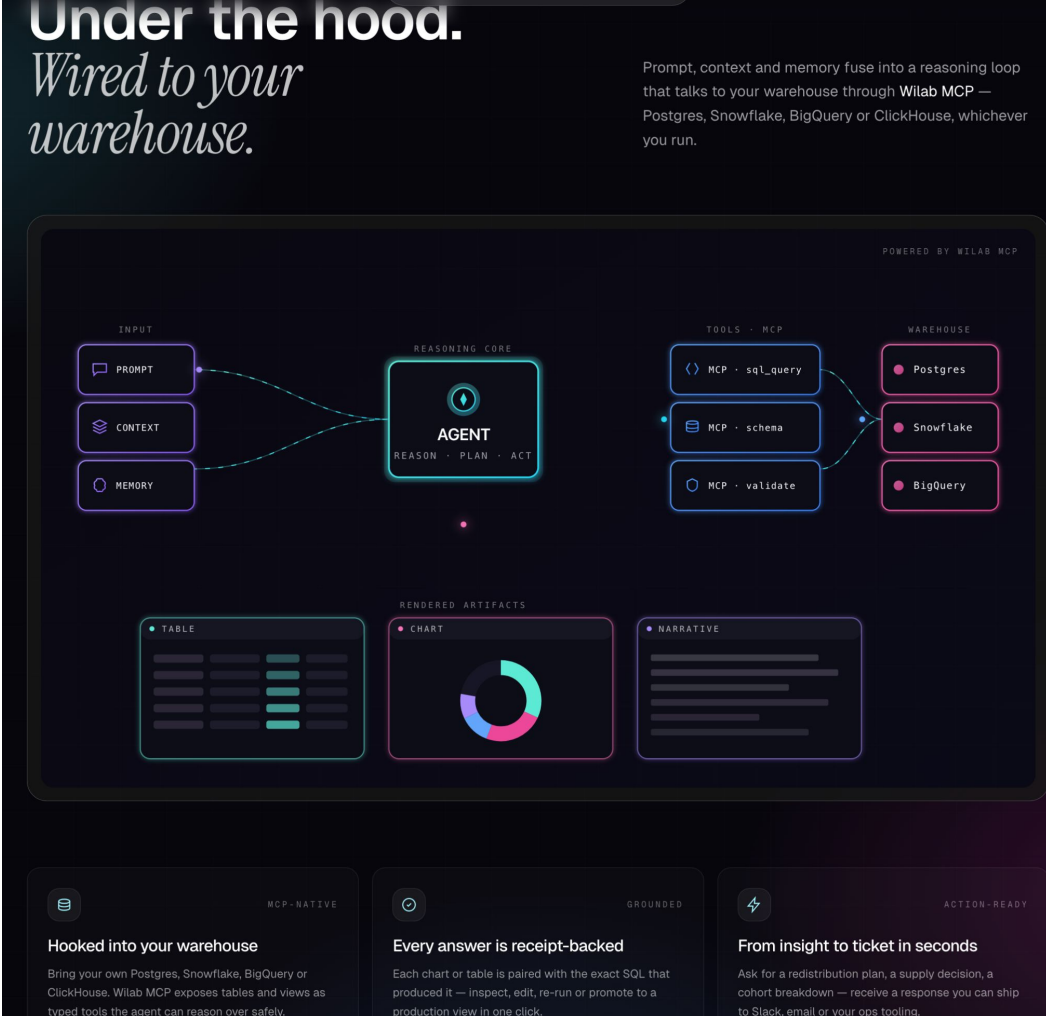


Diagrama 2 ("Not prompts. Context"): Es el plano interno del **motor** (cómo se gestiona la memoria, cómo se cargan las herramientas y cómo se optimizan los recursos computacionales para que el viaje no sea lento ni costoso para la empresa)

Mientras que el primer gráfico mostraba el flujo general de la información (el recorrido del dato), este diagrama, titulado **"Not prompts. Context."** (No instrucciones. Contexto), se enfoca exclusivamente en la **Ingeniería de Contexto** (*Context Engineering*), que es la disciplina técnica que Wilab utiliza para gestionar el cerebro del agente sin saturar la memoria del sistema.

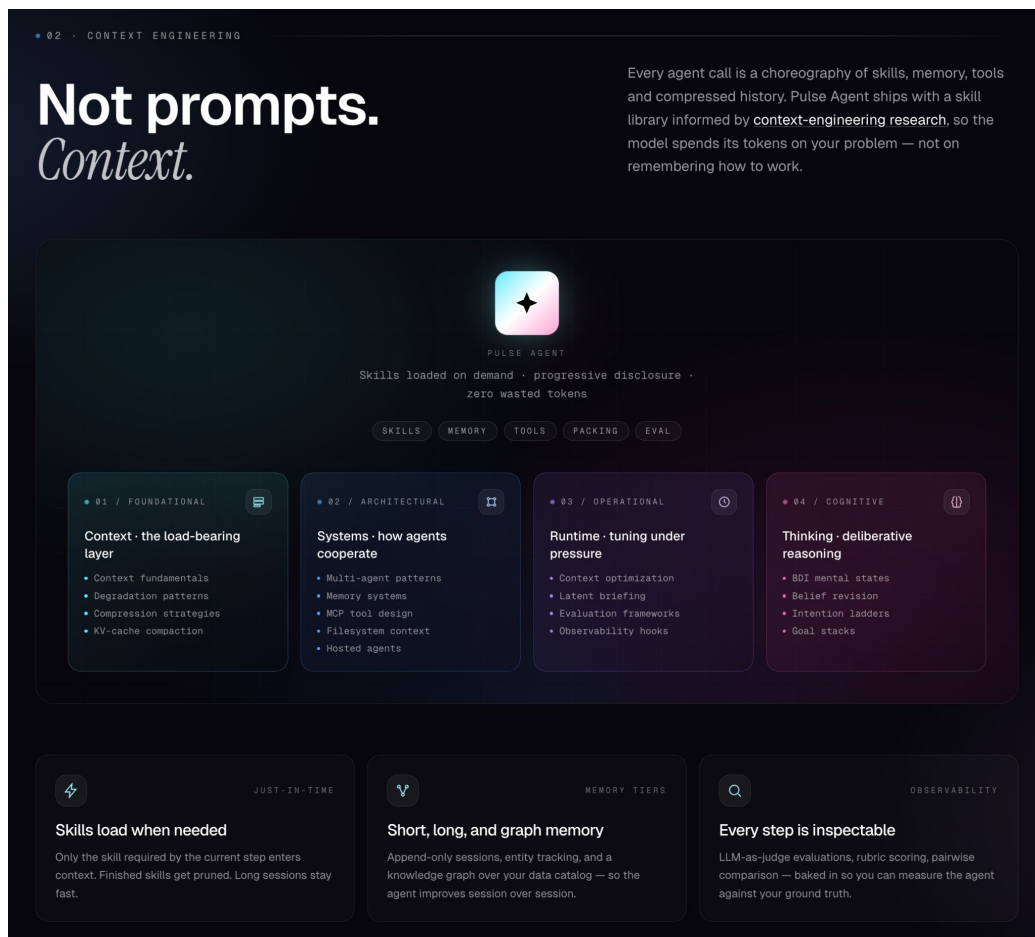
A continuación, se detalla la explicación del diagrama por secciones:

El Concepto Central (La sección superior)

El texto explicativo de la parte superior advierte sobre un problema recurrente en las arquitecturas de inteligencia artificial: si obligas al modelo a gastar su capacidad de memoria en "recordar cómo tiene que trabajar", desperdiciará recursos. Wilab propone una **"coreografía de habilidades, memoria, herramientas e historial comprimido"** para que el modelo consuma sus tokens de procesamiento únicamente en resolver tu problema de negocio.

En el centro del mapa conceptual se ubica el nodo del **Pulse Agent** con su filosofía operativa principal :

- **Skills loaded on demand** (Habilidades cargadas bajo demanda).
- **Progressive disclosure** (Revelación progresiva de información técnica).
- **Zero wasted tokens** (Cero tokens desperdiciados en el procesamiento de lenguaje).



Las 4 Capas de la Ingeniería de Contexto (Los bloques centrales del 01 al 04)

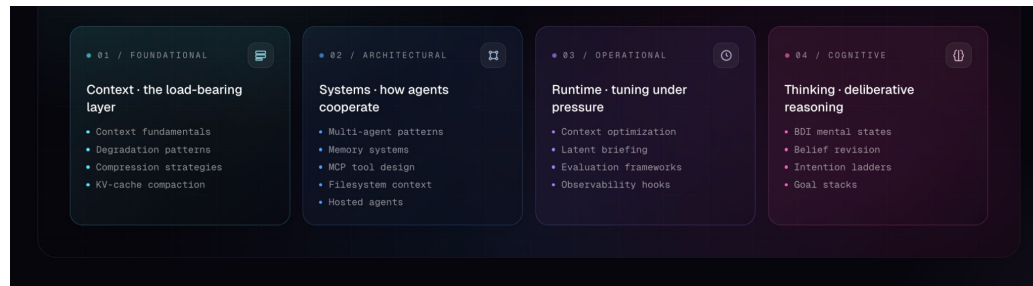
El núcleo del diagrama divide la ingeniería de contexto en cuatro dimensiones complementarias que van desde lo puramente técnico hasta el razonamiento abstracto :

01 / FOUNDATIONAL (Capa de Soporte / Estructura Física)

- **Context - the load-bearing layer:** Es la base de soporte del sistema.
- Aquí se manejan los aspectos técnicos de bajo nivel para que el modelo no se sature ni disminuya su rendimiento a medida que la conversación se vuelve muy larga. Para lograrlo, implementa fundamentos de contexto, patrones de degradación, estrategias de compresión y la compactación de la caché clave-valor (*KV-cache compaction*).

02 / ARCHITECTURAL (Arquitectura de Cooperación)

- **Systems • how agents cooperate:** Define las reglas de interacción del ecosistema de software.
- Esta capa integra patrones multi-agente, el diseño de herramientas MCP (*MCP tool design*), el contexto del sistema de archivos, los agentes hospedados y, de manera central, **los sistemas de memoria (*Memory systems*)** que actúan como la infraestructura de almacenamiento.



03 / OPERATIONAL (Operación y Ajustes en Tiempo Real)

- **Runtime • tuning under pressure:** Cómo se comporta y calibra el agente en vivo cuando se le somete a consultas de alta demanda.
- Incluye optimización del contexto en tiempo de ejecución, resúmenes latentes (*latent briefing*), marcos de evaluación y ganchos de observabilidad (*observability hooks*).

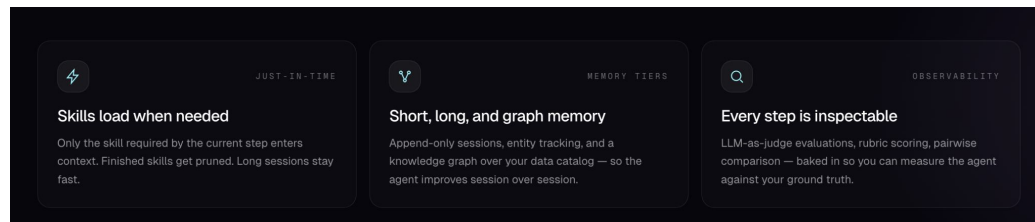
04 / COGNITIVE (Razonamiento Deliberativo o "Pensamiento")

- **Thinking • deliberative reasoning:** La estructura lógica con la que el agente toma decisiones antes de dar una respuesta.
- Utiliza el modelo cognitivo **BDI** (*Belief-Desire-Intention* o Creencia-Deseo-Intención), revisión de creencias (*belief revision*), escaleras de intención (*intention ladders*) y pilas de objetivos (*goal stacks*) para planificar paso a paso de forma lógica, imitando el razonamiento humano estructurado.

Las Tres Características Clave (La sección inferior del gráfico)

En la base del diagrama es donde finalmente se **desglosan e ilustran** de forma explícita las características que explicaban tu duda anterior :

1. **Skills load when needed (Just-In-Time / Carga a demanda):** Las habilidades analíticas del agente solo entran en su "memoria de trabajo" cuando el paso actual de la tarea lo requiere. Una vez que esa habilidad se usa (por ejemplo, escribir una consulta SQL específica), se elimina (*pruned*) de la ventana de contexto para que las sesiones largas sigan siendo rápidas y baratas.
2. **Short, long, and graph memory (Memory Tiers / Niveles de Memoria):** Aquí se detalla de forma clara la estructura de memoria que mencionábamos en la respuesta anterior. El bloque central inferior de este diagrama describe explícitamente los **tres niveles de memoria de Wilab** :
 - *Short memory:* Sesiones de chat de solo adición (*append-only sessions*) para la interacción del momento.



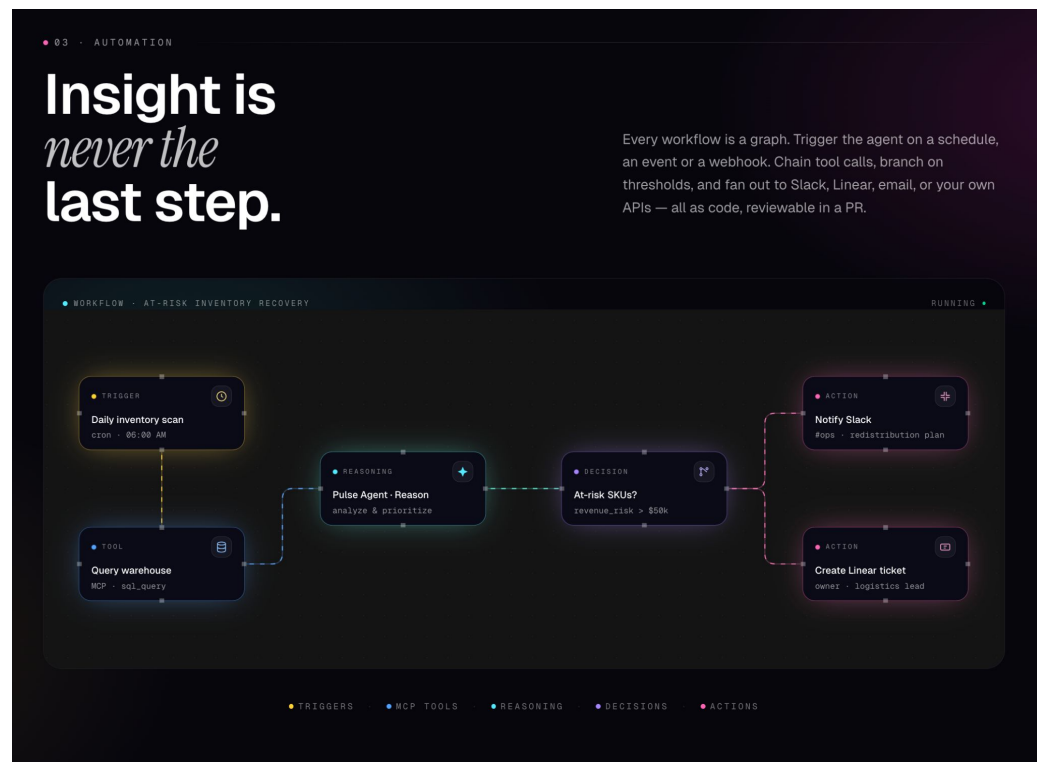
- *Long memory:* Rastreo de entidades (*entity tracking*) para recordar conceptos a lo largo del tiempo.
- *Graph memory:* Un **grafo de conocimiento** estructurado sobre tu propio catálogo de datos corporativos.

3 Every step is inspectable (Observabilidad total): Permite auditar cada razonamiento del agente mediante evaluaciones de "LLM-como-juez" (*LLM-as-judge*), puntuación de rúbricas y comparaciones emparejadas para medir el desempeño real de la IA contra la verdad del terreno (*ground truth*) de tu empresa.

Diagrama 3 (Automation): Muestra los **músculos**. Cómo pasa de responder preguntas en un chat a convertirse en un miembro activo de la empresa que vigila tus datos por la noche, toma decisiones y delega tareas en tus herramientas de trabajo diarias.

La filosofía de este gráfico es que de nada sirve que una inteligencia artificial te muestre una anomalía en un gráfico si luego un humano tiene que ir manualmente a resolverla. Por ello, Wilab permite estructurar **flujos de trabajo como grafos lógicos** (workflows) que ejecutan acciones autónomas en las herramientas diarias de la empresa (como Slack o Linear), gestionados completamente como código auditable en un PR (Pull Request).

El diagrama muestra un caso práctico real de un flujo de trabajo activo llamado **"at-risk inventory recovery"** (recuperación de inventario en riesgo), que se encuentra en estado de ejecución (*Running*). El flujo se lee de izquierda a derecha siguiendo la leyenda de colores de la parte inferior :



Paso a Paso del Flujo de Trabajo (Workflow)

1. El Disparador (TRIGGERS · Amarillo)

- **Daily inventory scan (cron · 06:00 AM):** El flujo no requiere que un humano escriba un prompt. En este caso, se activa de forma automática todos los días a las 6:00 AM mediante una tarea programada (cron).

2. La Herramienta de Consulta (MCP TOOLS · Azul)

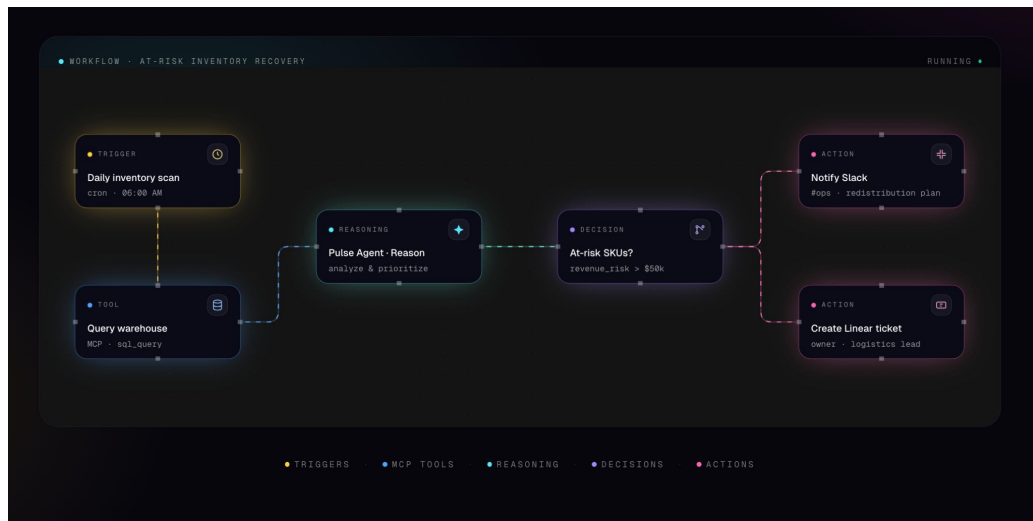
- **Query warehouse (MCP · sql_query):** Al activarse el disparador, el sistema ejecuta de inmediato una consulta en el almacén de datos (Postgres, Snowflake, etc.) utilizando la herramienta nativa de consulta SQL de Wilab basada en el protocolo MCP.

3. El Razonamiento (REASONING · Celeste/Cian)

- **Pulse Agent · Reason (analyze & prioritize):** Con los datos de inventario recuperados, el cerebro del agente entra en acción. Analiza la información recopilada de la base de datos para priorizar y encontrar dónde están los problemas críticos de stock o costes.

4. La Decisión Lógica (DECISIONS · Morado)

- **At-risk SKUs? (revenue_risk > \$50k):** El agente aplica una regla de negocio inteligente para tomar una decisión. En este paso, evalúa si los productos en riesgo (SKUs) representan una pérdida o riesgo financiero mayor a los USD 50,000.



5. Las Acciones Consecuentes (ACTIONS · Magenta)

Si la regla de decisión se cumple (es decir, el riesgo es mayor a la cantidad establecida), el flujo se bifurca de manera simultánea en dos acciones concretas para solucionar el problema de inmediato :

- **Notify Slack (#ops · redistribution plan):** Envía de forma automática una notificación al canal de operaciones de Slack con un plan de redistribución de inventario ya redactado por el agente.
- **Create Linear ticket (owner · logistics lead):** Crea automáticamente una tarea (ticket) en la plataforma de gestión de proyectos Linear, asignándole la resolución del problema directamente al líder de logística de la empresa.

Diagrama 4 (Gobernanza y Transparencia - SQL Studio): Le da el control final a los humanos (analistas y DBAs), permitiéndoles auditar el código SQL de la IA, modificarlo con un control de versiones y promoverlo a métricas de producción en un clic.

Esta última imagen, titulada **"04 · SQL STUDIO. The SQL behind every answer"** (El código SQL detrás de cada respuesta), ilustra la interfaz de control técnico y gobernanza de Wilab.

Mientras que los diagramas anteriores explicaban cómo se conecta el agente (Diagrama 1), cómo procesa la información (Diagrama 2) y cómo actúa automáticamente (Diagrama 3), esta imagen muestra el **entorno de desarrollo interactivo (Playground)** diseñado para los analistas, desarrolladores y administradores de bases de datos (DBAs). Su objetivo es eliminar el problema de la "caja negra" de la inteligencia artificial, permitiendo auditar, editar y poner en producción las consultas del agente sin procesos manuales.

A continuación, se detalla la explicación de este entorno paso a paso:

• 04 • SQL STUDIO

The SQL *behind every* answer.

A safe playground to explore sources and validate transformations. Craft queries in natural language, edit the result, promote them into production metrics with a single click — no copy-paste, no lost lineage.

Views (8)

> v2g3g

> v4g

timestamp node metric_name metric_value timestamp varchar varchar float

> vattach

> vdetach

> vepdg

> vsei

> vsessions

> vsuccess

Untitled-1.sql •

Untitled-2.sql •

+

```
1 SELECT node, MAX(metric_value) FROM v4g WHERE metric_name = 'cpuAttack' GROUP BY node;
2
3
4 WITH summed_sessions AS (
5   SELECT
6     timestamp,
7     SUM(metric_value) AS total_sessions
8   FROM sessions
9   WHERE metric_name = 'podStartTimestamp'
10  AND timestamp >= NOW() - INTERVAL '24 hours'
11  GROUP BY timestamp
12 }
13 SELECT
```

Query Results

Table

Chart

Hide

total_sessions: 943500838.00

Total

15062970973.11

15.4%

Pie Chart

Line Chart

Vertical Bar

Pie Chart

Powered by Wilab.io

izquierda)Esta sección describe cómo la herramienta tiende un puente seguro entre el lenguaje natural de negocio y el código técnico de base de datos :

- Version-safe edits (Ediciones seguras con control de versiones):** El código generado no se queda en el aire. Cada

El usuario simplemente describe la métrica que necesita en lenguaje conversacional (por ejemplo: *"Muestra las sesiones activas de las últimas 24 horas"*). El Studio se encarga de traducir esa intención, proponiendo la consulta SQL óptima y sugiriendo el tipo de gráfico adecuado para visualizarla.

3. **One-click promotion (Promoción en un solo clic):** Una vez que un analista humano verifica que la consulta SQL propuesta por la IA es correcta, puede "promoverla" a una vista de producción con un solo clic. No hace falta copiar y pegar código en otros sistemas; los tableros de control externos (*downstream dashboards*) de la empresa la adoptarán automáticamente.

[illegible]

- 4. Back-channel to the agent (Canal de retorno al agente):** Existe una conexión bidireccional. Si un usuario está interactuando con el chat conversacional del Pulse Agent y ve un gráfico, puede hacer clic en él para abrir la consulta SQL subyacente en el Studio, editarla, volverla a ejecutar o corregirla sobre la marcha.

Análisis de la Interfaz Visual (Mockup de la derecha)

La pantalla del dispositivo muestra exactamente cómo se ve la herramienta en funcionamiento en un entorno real de datos de red :

- **El Panel de "Views" (Izquierda):** Muestra el catálogo de vistas de datos activas (como `v4g`, `vsuccess`, `vsessions`). Al seleccionar una de ellas (en este caso, `v4g`), se despliega su esquema real con los tipos de datos exactos de sus columnas (`timestampgen timestamp`, `node varchar`, `metric_name varchar`, `metric_value float`). Esto demuestra visualmente cómo el protocolo MCP (*Model Context Protocol*) expone la estructura de la base de datos de manera fuertemente tipada para que el agente trabaje con ella.
- **El Editor de SQL (Centro-Derecha):** Se aprecian las pestañas de trabajo (`Untitled-1.sql`, `Untitled-2.sql`) con código SQL real y estructurado. La consulta utiliza funciones avanzadas y temporales (como `MAX(metric_value)`), operaciones de filtrado `WHERE metric_name = 'apsAttach'` y agrupaciones lógicas `GROUP BY`). Esto demuestra que el agente es capaz de estructurar consultas complejas con agregaciones reales.
- **Resultados de la Consulta (Abajo):** Se divide en dos pestañas: *Table* (Tabla con datos crudos) y *Chart* (Gráfico). En la imagen se encuentra activa la pestaña *Chart*, mostrando un gráfico de dona (*Pie Chart*) que renderiza de manera interactiva los resultados de la consulta SQL. A la derecha, un menú desplegable permite al usuario cambiar el tipo de visualización de forma inmediata (entre *Pie Chart*, *Line Chart* o *Vertical Bar*).

